FlashAttention Accelerator on PYNQ-Z2

Hardware Design — NYU Spring 2026

Anuj Apte ▪ Ricardo Díaz ▪ Raquel Brown
New York University

Outline

The Memory Bottleneck in Standard Attention

Standard scaled dot-product attention:

$$O = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

The $N \times N$ score matrix QK^T must be:

- Fully materialized before softmax
- Written to memory, then read back
- $4N^2$ bytes (float32)

For N=512: score matrix $\approx$ **1 MB**.

PYNQ-Z2 has only 140 BRAMs — it doesn't fit.

Our approach: FlashAttention

Process Q, K, V in small tiles.
Maintain running softmax state *on-chip*.

Peak memory: $O(N \cdot d)$ **not** $O(N^2)$

This project

- Vitis HLS implementation
- Target: **PYNQ-Z2** (Zynq XC7Z020)
- 4-channel AXI4 + AXI4-Lite control
- Verified against float64 golden model

Standard Attention — Numerical Example (N=3, d=2)

$$Q = K = V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad s_{ij} = \frac{q_i \cdot k_j}{\sqrt{2}}$$

Full 3×3 score matrix (all 9 values in memory)

	k_1	k_2	k_3
q_1	0.71	0.00	0.71
q_2	0.00	0.71	0.71
q_3	0.71	0.71	1.41

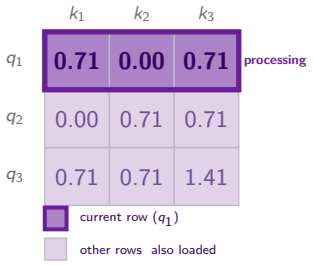
Softmax row q_1:

$e^{[0.71,\ 0.00,\ 0.71]-0.71} = [1.00,\ 0.49,\ 1.00]$

$\ell = 2.49 \ \Rightarrow \ p_1 = [0.40,\ 0.20,\ 0.40]$

$o_1 = p_1 \cdot V =$ **[0.80, 0.60]**

All N^2=9 scores in memory at once



Softmax needs all 9 scores before it can compute a single probability.

FlashAttention — Same Example, $B_Q{=}B_K{=}2$

Focus on row $q_1{=}[1,0]$. Init: $m{=}-\infty$, $\ell{=}0$, $o{=}[0,0]$.

K/V tile 0 (k_1, k_2 on-chip)

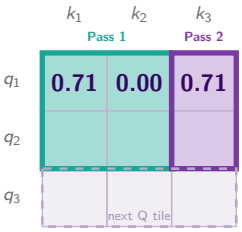
- Scores: $[0.71,\ 0.00]$
- $m \leftarrow 0.71, \quad e^{s-m}{=}[1.00,\ 0.49]$
- $\ell \leftarrow 1.49$
- $o \leftarrow \frac{1.00\cdot[1,0]+0.49\cdot[0,1]}{1.49} = [0.67,\ 0.33]$

K/V tile 1 (k_3 on-chip)

- Score: $[0.71]$ (m unchanged $\Rightarrow$ correction $c{=}1$)
- $\ell \leftarrow 1 \cdot 1.49 + 1.00 = 2.49$
- $o \leftarrow \frac{1\cdot 1.49\cdot[0.67,0.33]+1.00\cdot[1,1]}{2.49}$
  $= \mathbf{[0.80,\ 0.60]}$ ✓**same answer**

Max on-chip at once: $1{\times}2 = 2$ **scores.**
Full $3{\times}3$ **matrix never materializes.**

$3{\times}3$ score space Q tile 0 (q_1, q_2)



Numbers shown are q_1's scores per tile.
Each pass loads only one K/V tile at a time.

Online Softmax: Exact Attention Without the Full Matrix

State per query row i (lives on-chip):

- m_i — running row maximum
- l_i — running normalization sum
- o_i — running output accumulator

Per K/V tile update:

$$m_i^{\text{new}} = \max\left(m_i,\ \max_j s_{ij}\right)$$

$$l_i^{\text{new}} = \underbrace{e^{m_i - m_i^{\text{new}}}}_{\text{correction } c} l_i + \sum_j e^{s_{ij} - m_i^{\text{new}}}$$

$$o_i^{\text{new}} = \frac{c\, l_i\, o_i + \sum_j e^{s_{ij} - m_i^{\text{new}}} v_j}{l_i^{\text{new}}}$$

o_i stays normalized after every tile.
No final division needed.

Algorithm (BQ = BK = 8)

```
for q0 in 0, BQ, 2×BQ, ..., N:
  load Q tile                                   (8 rows)
  init m=−∞, l=0, o=0
  for k0 in 0, BK, 2×BK, ..., N:
    load K, V tiles
    scores = Q_t K_t^T / √d
    apply causal mask if needed
    update m, l, o online
  write output tile
```

- Correction $c = e^{m_i - m_i^{\text{new}}}$ rescales prior state when max grows
- Causal mask: $s_{ij} = -10^{30}$ for $j > i$, so $e^{s_{ij}} \to 0$ with no extra branch

IP Specification

Compile-time limits

`N_MAX`	64	max seq length
`D_MAX`	32	max dimension
`BQ`	8	Q tile size
`BK`	8	K/V tile size

Runtime (AXI4-Lite regs)

`N`	sequence length
`d`	embedding dim
`causal`	0 or 1
Q/K/V/O addr	64-bit DDR ptr

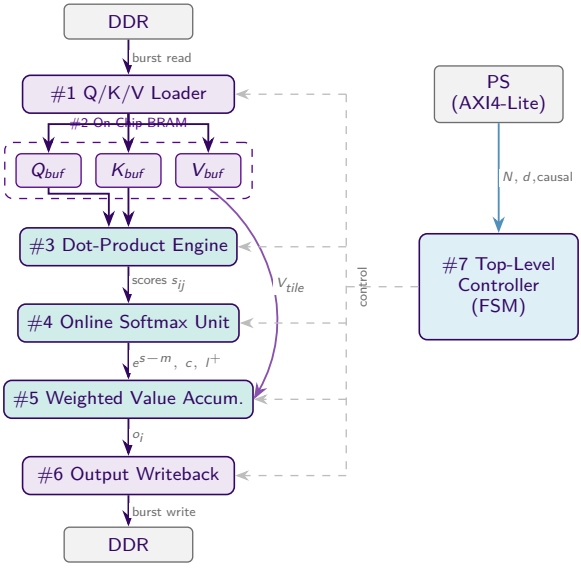
Hardware / Software boundary

- **PS (ARM):** allocates DDR buffers; writes parameters to AXI4-Lite registers; asserts `AP_START`; polls `AP_DONE`
- **PL (this IP):** reads tiles via AXI4 Master, computes on-chip, writes O back to DDR
- Intermediate state (m, l, scores, acc) *never* leaves the chip

AXI4 Master channels (no DDR contention)

`gmem0`	Q	read	32-bit / 16-burst
`gmem1`	K	read	32-bit / 16-burst
`gmem2`	V	read	32-bit / 16-burst
`gmem3`	O	write	32-bit / 16-burst

Architecture Overview



HLS Implementation Highlights

Dot-product engine: II = 1 via full unroll

```
SCORE_LOOP_J:
for (int j = 0; j < k_lim; j++) {
#pragma HLS PIPELINE II=1
  data_t dot = 0.0f;
  for (int x = 0; x < D_MAX; x++) {
#pragma HLS UNROLL    // 32 parallel MACs
    if (x < d) dot += Qbuf[i][x]*Kbuf[j][x
        ];
  }
  float s = dot * inv_sqrt_d;
  // causal: exp(-1e30) -> 0, no branch in
      softmax
  if (causal && (k0+j) > (q0+i)) s = -1
      e30f;
  scores[i][j] = s;
}
```

BRAM partitioned for 4-wide parallel reads

```
#pragma HLS ARRAY_PARTITION \
        variable=Qbuf cyclic factor=4 dim
            =2
#pragma HLS ARRAY_PARTITION \
        variable=Kbuf cyclic factor=4 dim
            =2
```

All inner loops: II = 1

Loop	II	Latency
LOAD_Q	1	12 cyc
LOAD_KV	1	12 cyc
SCORE_LOOP	1	202 cyc
FIND_MAX	1	3 cyc
UPDATE_ACC	1	34 cyc
STORE_O	1	9 cyc

Interface notes

- `float *Q` (1D) for AXI4 Master 2D arrays unsynthesizable
- `int causal` not `bool` 32-bit AXI4-Lite register
- `hls::expf/sqrtf` synthesizable math

Functional Verification — Testbench

Testbench pipeline

1. Python golden model generates vectors in **float64**
2. CSV loaded by C++ testbench (1D arrays, D_MAX stride)
3. `flash_attention_hls()` called in C-sim *and* RTL co-simulation identically
4. Max absolute error reported per test

Coverage

- Single-tile ($N \leq 8$)
- Multi-tile (N=16, N=32)
- **Partial tile** (N=13, q_lim = k_lim = 5)
- Max size (N=64, d=32)
- Full and causal masking
- Verified in Vitis HLS (C-sim & RTL co-sim)

Functional Verification — Results

Per-test results (Vitis HLS C-sim & RTL co-sim, tolerance $= 10^{-4}$)

ID	N	d	Causal	Max $\lvert\text{err}\rvert$ vs float64	
0	4	4	No	6.0×10^{-8}	✓
1	4	4	Yes	1.2×10^{-7}	✓
2	8	4	Yes	1.2×10^{-7}	✓
3	16	8	No	2.1×10^{-7}	✓
4	16	8	Yes	1.2×10^{-7}	✓
5	13	8	No	3.6×10^{-7}	✓
6	13	8	Yes	2.4×10^{-7}	✓
7	32	16	No	1.8×10^{-7}	✓
8	64	32	No	3.0×10^{-7}	✓

9 / 9 PASS

Max error $\mathbf{3.6 \times 10^{-7}}$

$270\times$ below 10^{-4} tol.

C-sim and RTL co-sim results are identical.

Test 5 (N=13) is the worst case partial tile boundary where the last Q and K tiles are only 5 rows wide.

Synthesis Results — PYNQ-Z2 (XC7Z020, 100 MHz)

Resource Utilization

Resource	Used	Avail	%
BRAM_18K	141	280	50%
DSP48E1	178	220	**81%**
Flip-Flops	31 527	106 400	30%
LUTs	40 259	53 200	**76%**

Key driver: 32 fully-unrolled MACs consume **141 of 178 DSPs**.

Switching to `ap_fixed<16,6>` would cut DSP usage significantly.

Timing

Metric	Value
Target clock	10 ns (100 MHz)
Timing slack	-1.40 **ns**
Critical path	FIND_MAX chain
Est. max freq	$\approx$ 87 MHz

AXI4 burst inference

- All 4 M_AXI ports: **burst inferred**
- Variable-length, up to tile size
- 4 separate DDR channels = no contention

Conclusions & Future Work

Delivered

- Full FlashAttention forward pass, Vitis HLS
- PYNQ-Z2 (XC7Z020) — PS-ready IP block
- 4-channel AXI4 Master + AXI4-Lite control
- All inner loops: **II = 1**
- **9/9 tests pass** in Vitis HLS (C-sim & RTL co-sim)
- Max error: 3.6×10^{-7} (270× below tolerance)
- Optional causal masking at no latency cost

Next steps

- Registered FIND_MAX tree → fix timing
- `#pragma HLS DATAFLOW` → overlap load / compute / store
- `ap_fixed<16,6>` → cut DSPs, quantify accuracy tradeoff
- Multi-head via time-multiplexing

Thank you

Questions?

Anuj Apte · Ricardo Díaz · Raquel Brown

github.com/ricardotk002/hwdesign-flash-attn-accel